

Task 1.1: Trace Bubble Sort Logic

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True  
        if not swapped:  
            break  
    return arr
```

Trace Bubble Sort manually on the array: `arr = [5, 2, 9, 1, 5, 6]`

```

start          [5, 2, 9, 1, 5, 6]

-- i=0 (j runs 0..4) --
i=0 j=0  5 > 2   SWAP   [2, 5, 9, 1, 5, 6]
i=0 j=1  5 <= 9   keep   [2, 5, 9, 1, 5, 6]
i=0 j=2  9 > 1   SWAP   [2, 5, 1, 9, 5, 6]
i=0 j=3  9 > 5   SWAP   [2, 5, 1, 5, 9, 6]
i=0 j=4  9 > 6   SWAP   [2, 5, 1, 5, 6, 9]    <- 9 locked at index 5

-- i=1 (j runs 0..3) --
i=1 j=0  2 <= 5   keep   [2, 5, 1, 5, 6, 9]
i=1 j=1  5 > 1   SWAP   [2, 1, 5, 5, 6, 9]
i=1 j=2  5 <= 5   keep   [2, 1, 5, 5, 6, 9]    <- equal, no swap (stable)
i=1 j=3  5 <= 6   keep   [2, 1, 5, 5, 6, 9]    <- 6 locked at index 4

-- i=2 (j runs 0..2) --
i=2 j=0  2 > 1   SWAP   [1, 2, 5, 5, 6, 9]
i=2 j=1  2 <= 5   keep   [1, 2, 5, 5, 6, 9]
i=2 j=2  5 <= 5   keep   [1, 2, 5, 5, 6, 9]    <- 5 locked at index 3

-- i=3 (j runs 0..1) --
i=3 j=0  1 <= 2   keep   [1, 2, 5, 5, 6, 9]
i=3 j=1  2 <= 5   keep   [1, 2, 5, 5, 6, 9]
no swaps in pass i=3 -> swapped stays False -> break

final          [1, 2, 5, 5, 6, 9]

Total comparisons: 14
Total swaps:      6

```

1.2 Insertion Sort Logic and Invariants

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j] # Shift right  
            j -= 1  
        arr[j + 1] = key        # Place key  
    return arr
```

Trace Insertion Sort manually on the array: `arr = [7, 3, 5, 8, 2]`

start [7, 3, 5, 8, 2] sorted prefix [7]

(key is pulled out into a variable, so the array shows a duplicate value while shifting until the key is placed back in)

-- i=1 key=3 (j starts at 0) --

i=1 j=0 7 > 3 shift 7 right [7, 7, 5, 8, 2]

i=1 j=-1 j < 0, stop

 place key at j+1=0 [3, 7, 5, 8, 2] <- prefix [3, 7] (1 cmp,

-- i=2 key=5 (j starts at 1) --

i=2 j=1 7 > 5 shift 7 right [3, 7, 7, 8, 2]

i=2 j=0 3 <= 5 stop

 place key at j+1=1 [3, 5, 7, 8, 2] <- prefix [3, 5, 7] (2 cmp,

-- i=3 key=8 (j starts at 2) --

i=3 j=2 7 <= 8 stop

 place key at j+1=3 [3, 5, 7, 8, 2] <- prefix [3, 5, 7, 8] (1 cmp,

-- i=4 key=2 (j starts at 3) --

i=4 j=3 8 > 2 shift 8 right [3, 5, 7, 8, 8]

i=4 j=2 7 > 2 shift 7 right [3, 5, 7, 7, 8]

i=4 j=1 5 > 2 shift 5 right [3, 5, 5, 7, 8]

i=4 j=0 3 > 2 shift 3 right [3, 3, 5, 7, 8]

i=4 j=-1 j < 0, stop

 place key at j+1=0 [2, 3, 5, 7, 8] <- prefix [2, 3, 5, 7, 8] (4 cmp,

final [2, 3, 5, 7, 8]

Total comparisons: 8

Total shifts: 6

Task 1.3: Sorting Analysis Questions

TODO 1.3A (Inversions & Shifts):

List all inversions (pairs of indices (i, j) where $i < j$ and $\text{arr}[i] > \text{arr}[j]$) in the initial array `[7, 3, 5, 8, 2]`:

A:

- Inversions: checked all 10 pairs with $i < j$ in `[7, 3, 5, 8, 2]` (indices `0..4`)
 - `(0,1): 7 > 3`
 - `(0,2): 7 > 5`
 - `(0,4): 7 > 2`
 - `(1,4): 3 > 2`
 - `(2,4): 5 > 2`
 - `(3,4): 8 > 2`
- Total number of inversions: 6
- Does this total exactly equal the number of shifts you counted in Task 1.2? (Yes/No):

TODO 1.3B (Early Stopping Flag):

Why does Bubble Sort require an explicit boolean flag (``swapped``) to achieve

$O(N)$ best-case time on sorted data, whereas Insertion Sort naturally achieves $O(N)$ with

A: Insertion Sort doesn't need a flag because if its all in order, then `arr[j]>key` is a so the while loop can finish immediately, and the for loop can therefore finish in $O(n)$ the same arr. Bubble sort will keep going without a flag even whens sorted, because the and will continue all the way until the end, checking everything even if already sorted

TODO 1.3C (Stability):

If a programmer changes line 33 of Bubble Sort to ``if arr[j] >= arr[j + 1]:``,

does the algorithm still produce a sorted array? Does it remain stable? Explain why or

A: Still sorted: Yes. Swapping two equal values does not change the order of the values and anything bigger than its neighbor still gets swapped right, so the largest element still bubbles to the end on every pass and the final array is in order.

Still stable: No. With `>=` two equal neighbors now swap places. Example with tags:

`[5a, 5b] -> 5a >= 5b` is True -> swap -> `[5b, 5a]`. The equal elements crossed each other so their original left-to-right order is lost.

Part 2: Lomuto Partition Scheme

Task 2.1: Trace Lomuto Partition Scheme

Trace Lomuto partition on `arr = [2, 8, 7, 1, 3, 5, 6, 4]` with `low = 0, high = 7` (Pivot = $arr[7] = 4$).

```
start    [2, 8, 7, 1, 3, 5, 6, 4]    pivot = 4    i = -1

j=0  arr[0]=2  2 <= 4  yes  i -> 0, swap arr[0],arr[0]    [2, 8, 7, 1, 3, 5, 6, 4]
j=1  arr[1]=8  8 <= 4  no   i stays 0                    [2, 8, 7, 1, 3, 5, 6, 4]
j=2  arr[2]=7  7 <= 4  no   i stays 0                    [2, 8, 7, 1, 3, 5, 6, 4]
j=3  arr[3]=1  1 <= 4  yes  i -> 1, swap arr[1],arr[3]    [2, 1, 7, 8, 3, 5, 6, 4]
j=4  arr[4]=3  3 <= 4  yes  i -> 2, swap arr[2],arr[4]    [2, 1, 3, 8, 7, 5, 6, 4]
j=5  arr[5]=5  5 <= 4  no   i stays 2                    [2, 1, 3, 8, 7, 5, 6, 4]
j=6  arr[6]=6  6 <= 4  no   i stays 2                    [2, 1, 3, 8, 7, 5, 6, 4]
end    swap arr[i+1]=arr[3] with arr[high]=arr[7]         [2, 1, 3, 4, 7, 5, 6, 8]
return i + 1 = 3
```

Resulting Left Subarray (≤ 4): [2, 1, 3] (indices 0..2)

Resulting Pivot Index and Value: index 3, value 4

Resulting Right Subarray (> 4): [7, 5, 6, 8] (indices 4..7)

Task 2.2: Lomuto Duplicate Analysis

TODO 2.2A:

If `arr = [5, 5, 5, 5, 5]` is partitioned using Lomuto partition (`pivot = 5`):

- What will the final value of `i` be at the end of the loop?
- What index will the pivot end up at?
- What are the sizes of the two recursive subproblems passed to quicksort?

A: Every `arr[j]` is 5 and `5 <= 5` is True, so `i` goes up by one on every step of the loop (`j = 0, 1, 2, 3`). Each "swap" is `arr[i]` with itself, so the array never changes.

- Final `i = 3` (that is `n - 2`, one less than the last index).
 - Pivot ends up at `i + 1 = 4`, which is high, the very last index.
 - Left subproblem `arr[0..3]` has size 4 (`n - 1`). Right subproblem is empty, size 0.
- This is the worst possible split. Each level of recursion only removes one element, so quicksort takes `N` levels and $O(n^2)$ total time on identical elements.

TODO 2.2B:

Run ``python lomuto_partition.py`` to view DEMO 3.

How does Hoare partition partition the array `[5, 5, 5, 5, 5]`?

What are the resulting subproblem sizes?

A: Hoare uses two pointers moving toward each other, and it stops each pointer on an element that is equal to the pivot instead of skipping it. So on `[5, 5, 5, 5, 5]` both pointers stop immediately, swap (5 with 5, no visible change), and move inward:

swap #1: `arr[0]` and `arr[4]`

swap #2: `arr[1]` and `arr[3]`

The pointers cross at `i = 2, j = 2`, and the split index is 2.

- Left subproblem `arr[0..2]` has size 3.
 - Right subproblem `arr[3..4]` has size 2.
-

Part 3: Quicksort Benchmarks (1,000,000 Elements & Edge Cases)

Task 3.1 & 3.2: Benchmark Observations & Analysis

TODO 3.1:

Record your execution times for $n = 1,000,000$ random integers:

- Python Timsort: 0.115 s (1.0x)
- Lomuto (Random Pivot): 0.931 s (8.1x)
- Lomuto (Median-of-3): 0.799 s (6.9x)
- Lomuto (Last Element): 0.823 s (7.1x)
- Hoare (Two-Pointer): 0.758 s (6.6x)
- 3-Way Quicksort: 1.156 s (10.0x)

On random data all the quicksorts are in the same ballpark, since a random array gives roughly balanced splits no matter how the pivot is picked. Timsort wins because it is written in C, not because of a better big-O.

Edge case times ($n = 10,000$ unless noted):

Scenario	Lomuto (Last)	Lomuto (Random)	Lomuto (Med-of-3)	Hoare	3-Way	Timsort
Already Sorted	1.3326 s	0.0051 s	0.0036 s	0.0030 s	0.0063 s	0.0000 s
Reverse-Sorted	0.9268 s	0.0053 s	0.0059 s	0.0030 s	0.0064 s	0.0000 s
All Identical	1.1959 s	1.2041 s	1.1922 s	0.0035 s	0.0002 s	0.0000 s
Few Unique ($n=20,000$)	0.4847 s	0.4895 s	0.4984 s	0.0079 s	0.0019 s	0.0008 s
Nearly Sorted ($n=20,000$)	0.0554 s	0.0112 s	0.0158 s	0.0063 s	0.0131 s	0.0002 s

TODO 3.2A:

In the edge case benchmarks:

What happens to Lomuto with Last Pivot on an Already Sorted array? Why?

How does Random Pivot or Median-of-3 fix this?

A: It blows up. Lomuto with the last element as pivot took 1.33 s on a sorted array of only 10,000 elements, while random pivot took 0.005 s and median-of-3 took 0.004 s. That is about 300x slower for the same input.

Why: on a sorted array the last element is the biggest one. Every other element is $\leq$ the pivot, so i moves up on every step and the pivot ends up staying at the end. of recursion doing about n work each, so $O(n^2)$ instead of $O(n \log n)$.

Fix: random pivot picks a random element and swaps it to the end before partitioning. On a sorted array a random element is probably somewhere in the middle of the values so the split is roughly even. Median-3 is similar, on a sorted array it is the median.

TODO 3.2B:

On the "All Identical Elements" scenario:

Compare the runtime of Lomuto Quicksort vs. Hoare Quicksort vs. 3-Way Quicksort.

Explain why Hoare and 3-Way Quicksort avoid Lomuto's quadratic explosion on identical data.

A: Times on 10,000 identical elements:

- Lomuto (last pivot): 1.196 s
- Lomuto (random pivot): 1.204 s
- Lomuto (median-of-3): 1.192 s
- Hoare: 0.0035 s
- 3-Way: 0.0002 s

All three Lomuto versions are equally bad here. Changing the pivot does not help, because every element is still the pivot value. The test `arr[j] <= pivot` is always true.

Hoare avoids it because it stops both pointers on elements equal to the pivot and swaps them. On identical data the pointers each move one step at a time from both ends and meet in the middle meaning $O(n \log n)$.

3-Way avoids it even better. It splits the array into three parts: less than pivot, equal to pivot, greater than pivot. Only the less and greater parts get recursed on. On identical data everything lands in the equal part, both other parts are empty, and there is nothing left to recurse on. One pass and done, so $O(n)$.

TODO 3.2C:

Why does standard recursive Quicksort risk crashing with `RecursionError` in Python when sorting large unbalanced arrays, and how does tail-recursion elimination or small-side recursion prevent this?

A: Python limits how deep function calls can nest.

Normal quicksort recurses on both halves. When the splits are unbalanced ($n-1$ and 0), each recursive call only shrinks the problem by one, so the recursion depth is n . For $n = 10,000$ that is $10,000$ nested calls, way past the limit, so Python raises `RecursionError` before it finishes.

The fix is to only recurse on the smaller side, and handle the bigger side with a loop instead of a recursive call (the tail-recursion elimination).

The smaller side is always at most half the array, so every recursive call at least halves the size. That caps the depth at about $\log_2(n)$.

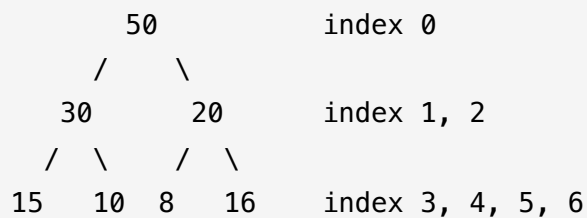
The loop does not use any stack space, so the big side is free.

This won't cause the recursion error to trigger.

Part 4: Representing Binary Trees via Arrays

Task 4.1: Manual Tree Traversal Trace

```
arr = [50, 30, 20, 15, 10, 8, 16] (0-indexed)
```



left child of $i = 2i + 1$

right child of $i = 2i + 2$

Pre-Order Traversal (Root → Left → Right): [50, 30, 15, 10, 20, 8, 16]

In-Order Traversal (Left → Root → Right): [15, 30, 10, 50, 8, 20, 16]

Post-Order Traversal (Left → Right → Root): [15, 10, 30, 8, 16, 20, 50]

Level-Order Traversal (Breadth-First): [50, 30, 20, 15, 10, 8, 16]

Is this array a valid Max-Heap? (Yes/No): [Yes]

Task 4.2: Array Tree Practice Code

Done in `array_tree_practice.py`.